EPSR-test-ex21 nwts: first-order differences and interactive corrections

Program(s): nwts | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises the two-sample path of `nwts`: the weights of each sample and of their first-order isotopic difference, in which only correlations involving the substituted atom types survive [1,2]. It also covers the input checks (option clamping, exchange flags, zero abundances) and the interactive recovery when the keyword file contains unknown mass numbers or chemical symbols.

Code exercised

Fortran (original, fortran_src/):

- `nwts.f`: `nwts` (samples 1 and 2, `<run>tot1/2.wts`, `<run>dif.wts`, `normtot`/`normdif`, per-molecule factor; stdin prompts for unknown mass numbers and symbols)
- `process_symbols.f`: `check_symbol` (`nwts` branch: `output` and `nsamples` clamped, `iexchange` forced to 0 for non-H atoms, `fnameato` fills default components)
- `ato_manipulation_rev4.f`: `scatt_len`; `read_and_save_files.f`: `read_input_file`, `read_ato`, `save_files_others`; `ato_read_and_save.f`: `minitc`, `moloutc`

Rust (port, rust/epsr-core/src/):

- `tools/nwts.rs`: `run`, `check_symbol_extra`, `read_isotopes`, `read_stdin_word`, `scatt_len`
- `symbols.rs`: `read_input_file`, `save_inp_file`; `ato.rs`: `read_ato`, `minitc`, `moloutc`

Physical system

The EPSR-test DMSO/water mixture (200 DMSO + 400 water, 3200 atoms, six atom types S, C, O, H, O1, H1) and a single water molecule (`WaterX.ato`). Hypothetical isotope-substitution experiments are set up: the methyl H as a 50/50 natural H/D mixture against pure 1H, together with the water H1 as 90 % D/10 % 1H against natural H in `FirstDiffA`, and 34S plus methyl H (D versus 1:1 H/D) substitution with an 18O entry of zero abundance in `FirstDiffB`. The weights use bound coherent scattering lengths [3] and Faber-Ziman pair products [4]; `normtot 1/2` divide by $\langle b \rangle^2$ or $\langle b^2 \rangle$, `output 2` gives weights per molecule (factor 3200/600 atoms per molecule).

Inputs

`FirstDiff{A,B,C}.NWTS.dat` made from `EPSR-test/DMSODWaterD.NWTS.dat` by changing keyword values; `WaterX.ato` is `EPSR-test/WaterMolecule.ato` with the symbol of O1 changed to `Ox`. Stdin: `diffc.stdin` gives the mass numbers 1 and 2 in place of 99 and 77, `waterx.stdin` gives `0` for `Ox`.

How the accuracy was verified

`commands.txt` (4 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

One pass of four `nwts` calls; 15 files compared (`tot1`, `tot2`, `dif` for each FirstDiff run, `WaterX.NWTStot.wts`, the rewritten `.NWTS.dat` files and `WaterX.ato`). The reference shows the checks acting: the rewritten `FirstDiffA`/`FirstDiffC` files carry `iexchange 0` for the oxygen, `FirstDiffB` carries `output 2` instead of 5, `WaterX.NWTS.dat` has been filled with two components from the `.ato` file, and `WaterX.ato` now has the symbol `O`.

Notes

As in the Fortran, the mass numbers typed on stdin are used for the weights but are not written back to `FirstDiffC.NWTS.dat`, which still lists 99 and 77.

References

[1] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).
[2] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[3] V. F. Sears, "Neutron scattering lengths and cross sections", Neutron News 3(3), 26-37 (1992).
[4] T. E. Faber and J. M. Ziman, "A theory of the electrical properties of liquid metals III. The resistivity of binary alloys", Phil. Mag. 11, 153-173 (1965).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.